

Lab 3 — Architecture Selection & Justification

Self-Service Coffee Kiosk System · PES1UG24CS127 · Software Engineering (UE23CS341A)

We chose **Layered (3-tier) Architecture** for the Self-Service Coffee Kiosk System.

Architectural Choice

A layered architecture organised as three tiers: a Presentation layer (the touch-screen User Interface component), a Business layer (Order Manager, Payment Service and Receipt Printer components), and a Data layer (the Menu & Pricing Data Store component). The whole stack is packaged and deployed as one self-contained unit running on the kiosk.

Two Reasons for the Choice

1. The kiosk is a single, self-contained node whose responsibilities divide cleanly along three lines: rendering the touch interface, executing order / payment / receipt logic, and storing menu and pricing data. A layered structure maps directly onto that separation, so each layer can be built, tested and replaced on its own: the Payment Service can be changed for a different card provider, or the Receipt Printer driver replaced for new hardware, with no change to the rest.
2. The requirement set is small and stable: three coffee types, two sizes, card-only payment, a printed receipt. Nothing in the scenario needs independent scaling or independent deployment of features, so the operational overhead, network latency and data-consistency problems of microservices would add cost with no benefit. Client-server fits no better: the scenario has no central server and no multiple clients, and it would add a single point of failure. Layered gives the modularity the system needs at the lowest complexity.

Security Advantage

The layer boundary gives sensitive data a single, well-defined path. The Presentation layer never handles card details and never touches the data store directly; every payment request passes through the Business-layer Payment Service component, which is the only component holding payment-gateway credentials and card-handling logic. This isolation shrinks the attack surface, since a compromised touch UI cannot read card data or run arbitrary database queries, and it keeps all PCI-relevant code in one place that can be audited and hardened.

Performance Benefit

Menu and pricing data are small and static, so the Data layer loads them into memory once at start-up and every order during a busy period is then served from cache with no repeated disk or database reads. Calls between layers are in-process method calls rather than network round-trips, which keeps per-order latency low and lets the kiosk clear a queue quickly.